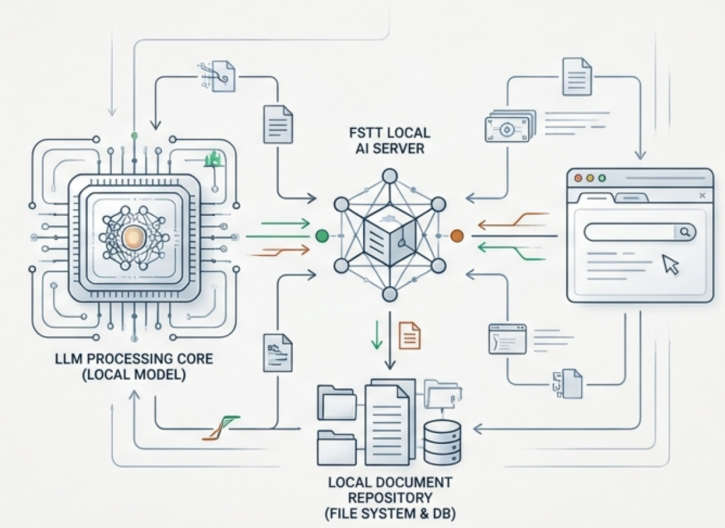
Practical Lab Report

Local AI with Ollama

Running and Developing Local AI Applications



Submitted by:
HACHEM ABDESSAMAD

PhD Student
Université Abdelmalek Essaâdi (UAE) / Faculté Polydisciplinaire de Larache (FPL)

October 26, 2026

Contents

1 Laboratory Screenshots **2**
1.1 Installation et Environnement 2
1.2 Intégration Python et Chatbot 3
1.3 Système RAG et Application Web 4

2 Questions for the Report **6**

1 Laboratory Screenshots

Voici les captures d'écran validant la réalisation des différentes étapes pratiques du laboratoire.

1.1 Installation et Environnement

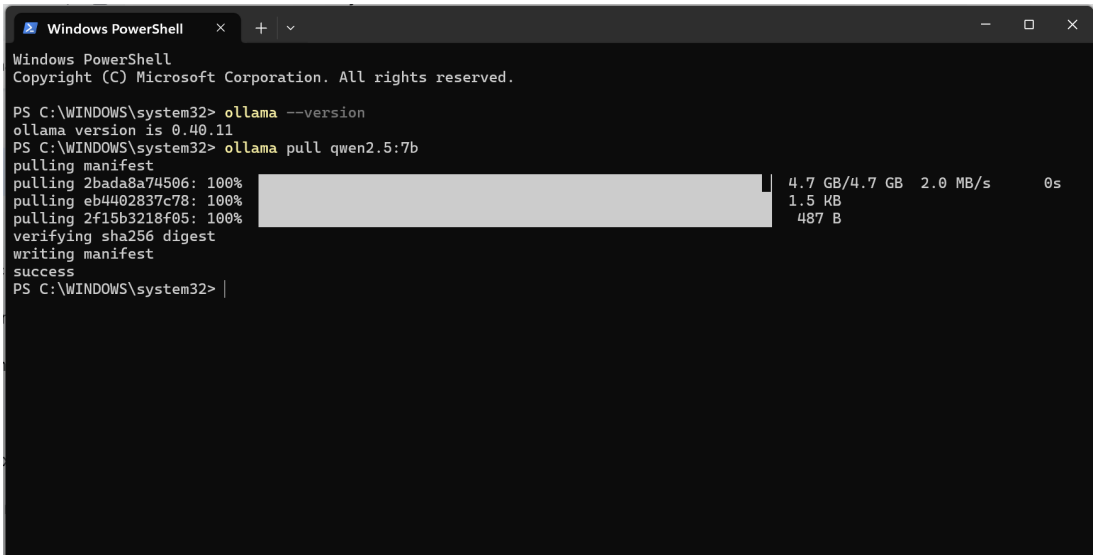


Figure 1: Installation réussie d'Ollama et vérification de la version via PowerShell.

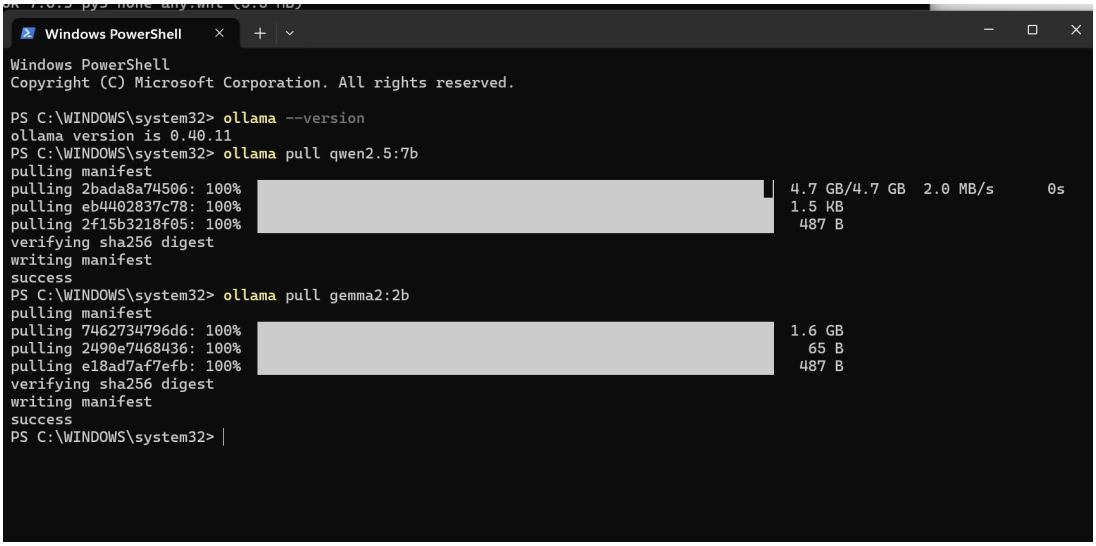


Figure 2: Gestion des modèles : liste des modèles téléchargés (qwen2.5:7b et gemma2:2b).

1.2 Intégration Python et Chatbot

[32]:
```python
history = []

def ask_ai_memory(question):
    history.append(f"User: {question}")
    prompt = "\n".join(history) + "\nAssistant:"
    data = {"model": MODEL, "prompt": prompt, "stream": False}
    response = requests.post(URL, json=data)
    response.raise_for_status()
    answer = response.json()["response"]
    history.append(f"Assistant: {answer}")
    return answer

def chat_loop_memory():
    while True:
        question = input("\nYou: ")
        if question.lower() == "exit":
            break
        print("\nAI:", ask_ai_memory(question))

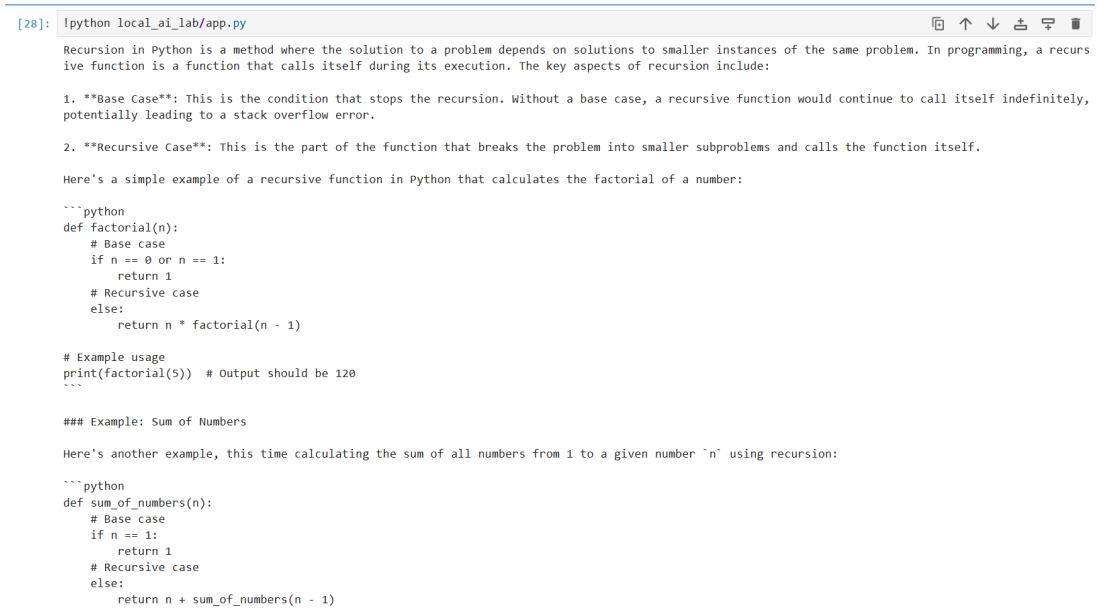
# chat_loop_memory()   # <- uncomment for interactive use
```

[33]:
```python
history.clear()
for q in ["My name is Ahmed.", "What is my name?"]:
    print("You:", q)
    print("AI :", ask_ai_memory(q), "\n")
```

```
You: My name is Ahmed.
AI : Hello Ahmed! Nice to meet you. How can I assist you today?

You: What is my name?
AI : Your name is Ahmed.
```

Figure 3: Test du chatbot dans Jupyter Notebook avec mémoire de conversation.



```
[28]: !python local_ai_lab/app.py

Recursion in Python is a method where the solution to a problem depends on solutions to smaller instances of the same problem. In programming, a recursive function is a function that calls itself during its execution. The key aspects of recursion include:

1. **Base Case**: This is the condition that stops the recursion. Without a base case, a recursive function would continue to call itself indefinitely, potentially leading to a stack overflow error.

2. **Recursive Case**: This is the part of the function that breaks the problem into smaller subproblems and calls the function itself.

Here's a simple example of a recursive function in Python that calculates the factorial of a number:

```python
def factorial(n):
    # Base case
    if n == 0 or n == 1:
        return 1
    # Recursive case
    else:
        return n * factorial(n - 1)

# Example usage
print(factorial(5))  # Output should be 120
```

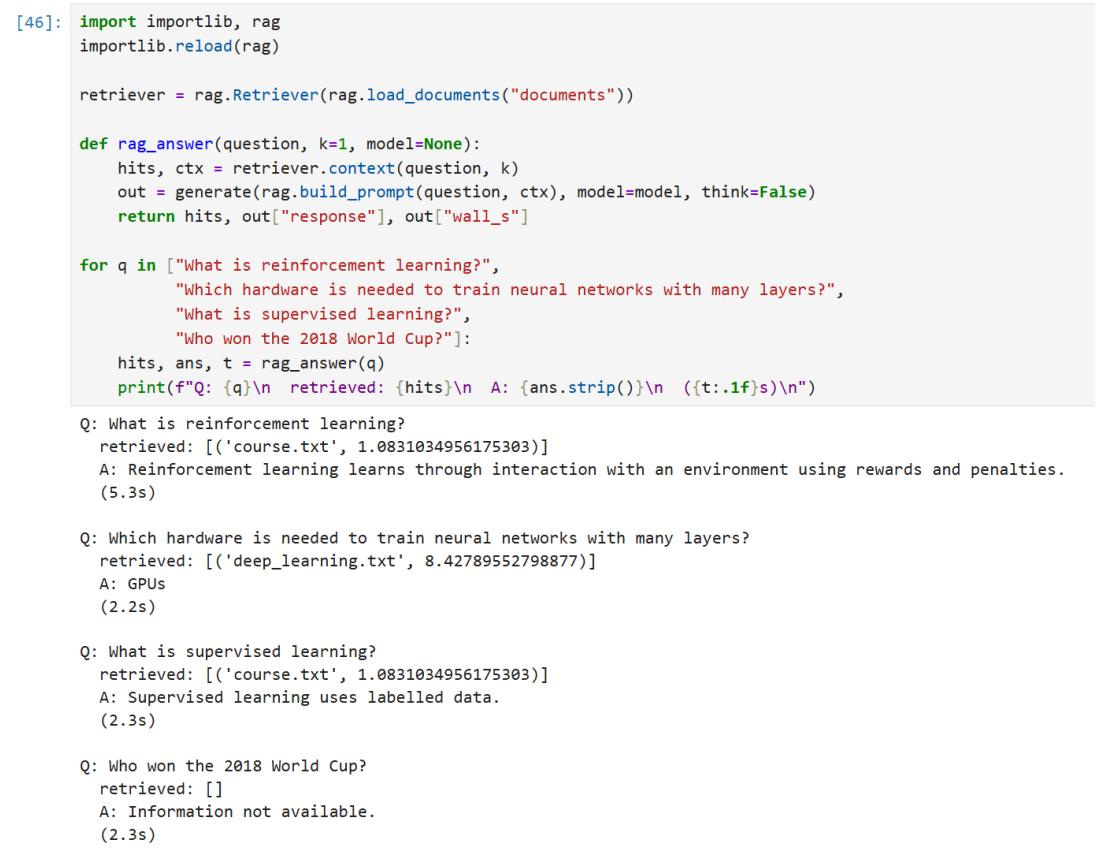
Example: Sum of Numbers

Here's another example, this time calculating the sum of all numbers from 1 to a given number `n` using recursion:

```python
def sum_of_numbers(n):
    # Base case
    if n == 1:
        return 1
    # Recursive case
    else:
        return n + sum_of_numbers(n - 1)
```
```

Figure 4: Exécution du script Python autonome (app.py) interrogeant l'API locale.

1.3 Système RAG et Application Web



```
[46]: import importlib, rag
      importlib.reload(rag)

      retriever = rag.Retriever(rag.load_documents("documents"))

      def rag_answer(question, k=1, model=None):
          hits, ctx = retriever.context(question, k)
          out = generate(rag.build_prompt(question, ctx), model=model, think=False)
          return hits, out["response"], out["wall_s"]

      for q in ["What is reinforcement learning?",
                "Which hardware is needed to train neural networks with many layers?",
                "What is supervised learning?",
                "Who won the 2018 World Cup?"]:
          hits, ans, t = rag_answer(q)
          print(f"Q: {q}\n  retrieved: {hits}\n  A: {ans.strip()}\n  ({t:.1f}s)\n")

Q: What is reinforcement learning?
  retrieved: [('course.txt', 1.0831034956175303)]
  A: Reinforcement learning learns through interaction with an environment using rewards and penalties.
  (5.3s)

Q: Which hardware is needed to train neural networks with many layers?
  retrieved: [('deep_learning.txt', 8.42789552798877)]
  A: GPUs
  (2.2s)

Q: What is supervised learning?
  retrieved: [('course.txt', 1.0831034956175303)]
  A: Supervised learning uses labelled data.
  (2.3s)

Q: Who won the 2018 World Cup?
  retrieved: []
  A: Information not available.
  (2.3s)
```

Figure 5: Démonstration du système RAG : récupération de documents pertinents et génération de réponses.

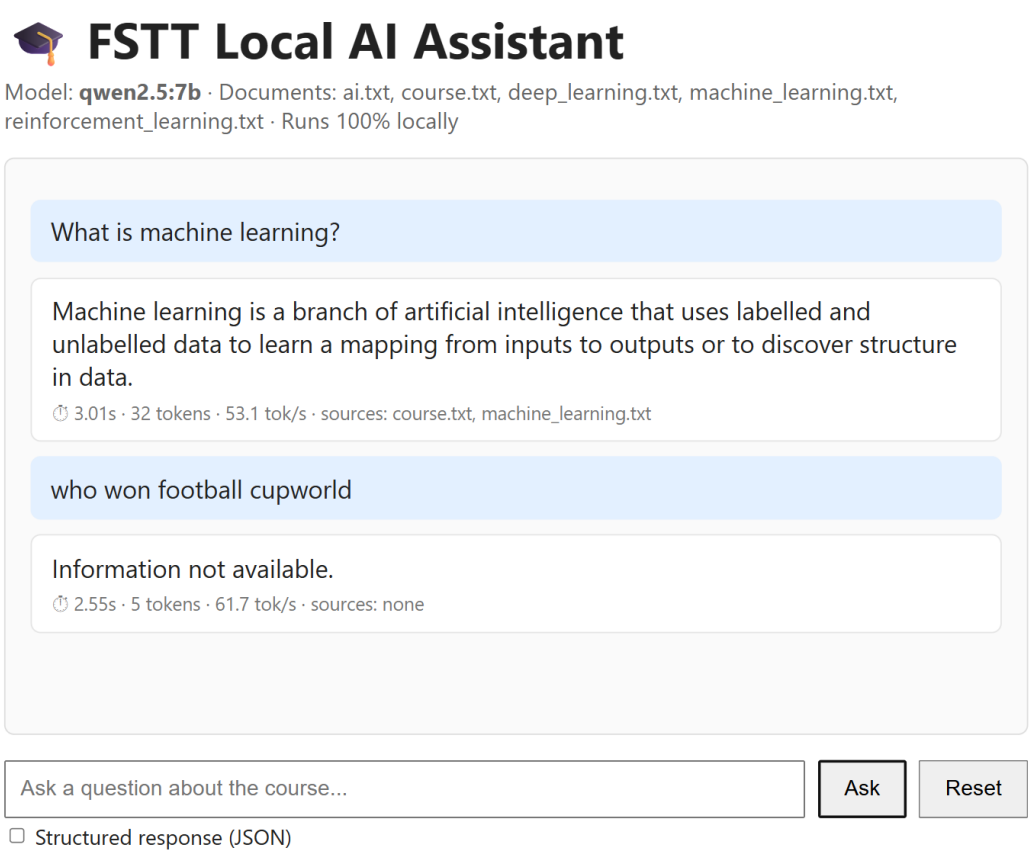


Figure 6: Interface de l'application web finale (FSTT Local AI Assistant) avec affichage des métriques.

2 Questions for the Report

Q1. What is Local AI?

Local AI refers to running artificial intelligence models directly on a user's personal device (computer, laptop, or smartphone) rather than relying on a remote cloud server. The data, computation, and model weights remain entirely on the local hardware.

Q2. Difference between a Cloud LLM and a Local LLM?

A Cloud LLM (like ChatGPT) runs on massive data centers. It requires an internet connection, sends your data to a third party, and usually charges per token, but offers state-of-the-art performance. A Local LLM runs on your own hardware (CPU/GPU). It guarantees 100% privacy, works offline, and is free to use, but is limited by your computer's RAM and processing speed.

Q3. What role does Ollama play?

Ollama acts as a local server and model manager. It downloads the models, optimizes them for your hardware (handling the GPU/CPU split), and exposes them via a REST API on `localhost:11434`. This allows any application (Python, Web apps) to easily communicate with the local LLM.

Q4. What is model quantization?

Quantization is a compression technique that reduces the size of the model. Instead of storing every weight (parameter) with 16-bit or 32-bit precision, it stores them using 4-bit or 8-bit precision (e.g., `Q4_K_M` seen in `ollama show`). This drastically reduces the RAM required to run the model, with a very small loss in accuracy.

Q5. Why does model size affect memory consumption?

A model's size (e.g., 7B parameters) determines how much RAM/VRAM is needed. Each parameter must be loaded into memory. A 7B model at 4-bit quantization requires about 4.7 GB of RAM just to load the weights, whereas a 2B model requires only 1.6 GB. Larger models need more memory to perform the mathematical operations.

Q6. Why can a smaller model sometimes be preferable?

As demonstrated in my benchmark, a smaller model like `gemma2:2b` is dramatically faster (111 tokens/s) and uses less than half the memory of a 7B model. A smaller model is preferable when: (1) the hardware is limited (e.g., 8 GB of RAM), (2) low latency is critical, and (3) the task is simple enough that advanced reasoning isn't necessary.

Q7. Advantages of local inference?

- **Absolute Privacy:** No data is sent to a third-party cloud. This is crucial for sensitive information (medical, legal, corporate data).

- **Offline Operation:** Works perfectly without an internet connection.
- **Zero Marginal Cost:** No API fees or subscription limits.
- **Full Control:** You own the model and the data pipeline.

Q8. Disadvantages of local inference?

- **Hardware Constraints:** Requires significant RAM/VRAM and a decent CPU/GPU.
- **Lower Capability:** Even the best 7B model is generally less "smart" than cloud giants like GPT-4.
- **Maintenance:** The user is responsible for downloading models, managing storage, and updating the server.
- **Speed:** Can be slower than cloud data centers without a dedicated high-end GPU.

Q9. Explain the architecture of your Python application.

The architecture follows a local client-server model:

1. The **Frontend** (HTML/JS) sends a question to the Flask server.
2. The **Flask Backend** uses a TF-IDF Retriever to search local `.txt` files and extract the most relevant context.
3. It constructs a prompt and sends an HTTP POST request to the **Ollama API** (`localhost:11434`).
4. **Ollama** runs the local LLM to generate the response.
5. The response is sent back to Flask and displayed in the web interface.

Q10. Explain the role of the REST API.

The REST API exposed by Ollama on `localhost:11434` acts as a standardized communication bridge. It allows our Python code to interact with the complex C++ inference engine using simple HTTP requests. By sending a JSON payload to endpoints like `/api/generate` or `/api/chat`, the application receives a structured JSON response with the generated text and performance metrics, cleanly decoupling the UI from the AI engine.

Q11. What is RAG?

RAG stands for **Retrieval-Augmented Generation**. It is an architectural pattern that enhances the accuracy of LLMs. A RAG system first retrieves relevant documents from a local knowledge base (using keyword or vector search). It then injects this retrieved context into the prompt sent to the LLM. The LLM is instructed to answer using *only* this context, preventing hallucinations and allowing the AI to answer based on specific, up-to-date, or private documents.

Q12. Difference between an LLM and a multimodal LLM?

A standard LLM is trained exclusively on text (input text, output text). A **multimodal LLM** (like LLaVA) is trained on multiple data types, typically text and images. It uses an image encoder to process pixels into embeddings that the language model can understand. This allows the model to describe images, read handwritten text, or answer questions based on visual input.

Q13. Compare two local models experimentally.

Based on the benchmark performed in the lab:

- **Speed:** `gemma2:2b` was significantly faster, generating 111.49 tokens/s compared to 50.33 tokens/s for `qwen2.5:7b`.
- **Memory:** `qwen2.5:7b` required 4.76 GB of VRAM, whereas `gemma2:2b` used only 1.93 GB.
- **Quality:** In the programming task, `qwen2.5:7b` provided a more optimized algorithm, proving that larger models generally have better reasoning capabilities. However, `gemma2:2b` still provided a perfectly functional answer.

Q14. Which model would you deploy on a computer with limited RAM? Explain.

I would deploy `gemma2:2b` (or an even smaller model). On a computer with limited RAM (e.g., 8 GB total), deploying `qwen2.5:7b` would consume nearly 5 GB just for the model weights, leaving very little memory for the OS and applications. This would cause "swapping" (using the hard drive as RAM), which drastically slows down the system. `gemma2:2b` uses only 1.9 GB, ensuring the system remains responsive.

Q15. What security risks exist when exposing a local LLM server to a network?

- **Unauthorized Compute Use:** By default, Ollama has no authentication. If exposed, anyone on the network can use your GPU.
- **Data Exposure:** Users could potentially extract sensitive data from the server's system prompts.
- **Prompt Injection:** Malicious users could craft prompts to bypass safety rules.
- **Denial of Service (DoS):** Massive requests could exhaust your RAM/VRAM, crashing your computer.

Mitigation: A secure deployment requires a reverse proxy (like Nginx) with Authentication and TLS/HTTPS encryption, or restricting access via a Firewall/VPN.